Shell Scripting — Complete Study Notes (Updated)

DevOps Training | Cognizant | Azure-Focused Environment | All 6 Phases + Bonus

PHASE 1 — THE MENTAL MODEL: Why Shell Scripting Exists

1.1 What is a Shell Script?

A shell script is a plain text file containing a series of Linux commands and shell statements. When executed, the shell reads and runs each command from top to bottom, line by line, exactly as if you had typed those commands into the terminal yourself. Shell scripts are written in a text editor — never a word processor like Microsoft Word, because word processors add invisible formatting characters that break script syntax.

Think of a shell script like a recipe. Instead of a chef executing each step manually every time, they write the recipe once and anyone can follow it to get the same result. The shell script is the recipe; the shell interpreter is the person cooking.

⚠ NOTE: Shell scripts use ASCII plain text only. Word processors turn straight quotes into curly quotes. Curly quotes break shell syntax. Always use a plain text editor: vim, nano, emacs, VSCode, Atom.

1.2 Why Write Shell Scripts?

Reason 1 — Automation of Repetitive Tasks

Any time you type the same set of commands more than once, a shell script lets you automate that sequence so it can be triggered with one command, or scheduled to run automatically at a specific time without you being present.

Reason 2 — Documentation of Rare Processes

Some tasks are performed rarely but are critical. Each time you do them manually you have to re-read documentation and re-teach yourself. A shell script encodes that knowledge permanently. It executes the right commands with the right options every single time, even when you are tired or under pressure at midnight.

Reason 3 — Delegation to Less-Experienced Staff

A shell script acts as a black box. You encode expert knowledge inside it. A helpdesk operator can execute the script without knowing any of the underlying details.

⚠ NOTE: Any task you can perform on the command line can be automated with a shell script. This is the single most important principle in this entire course.

1.3 Shell Scripts vs. Compiled Programs

Attribute	Shell Script	Compiled Program (C/C++)
Development Speed	Very fast — hours or minutes	Slow — days or weeks

Learning Curve	Low — if you know the CLI, you are almost there	High — pointers, types, memory
Performance	Slower execution	Extremely fast execution
Sweet Spot	Sysadmin, automation, reporting	High-performance applications
Data Types	No declaration needed	Must declare all types

1.4 Lab Environment — Vagrant and VirtualBox

The course uses VirtualBox as the virtualization platform and Vagrant as the automation layer. VirtualBox runs unmodified operating systems inside virtual machines. The physical computer is called the host; the virtual machine is called the guest.

Command	What It Does
vagrant box add jasonc/centos7	Downloads the OS image (box) to your local machine
vagrant init jasonc/centos7	Creates a Vagrantfile in the current directory
vagrant up	Starts all VMs defined in the Vagrantfile
vagrant up <vmname>	Starts only a specific named VM
vagrant ssh <vmname>	SSHs into a specific named VM
vagrant halt	Shuts down VMs without destroying them
vagrant reload	Restarts VMs and reloads Vagrantfile config
vagrant destroy	Permanently deletes the VM and all its data
vagrant status	Shows running/stopped state of all VMs

⚠ NOTE: vagrant destroy permanently deletes all data inside the VM. Work saved only inside the VM is lost. The /vagrant shared folder (mapped to your project folder on host) survives destruction.

PHASE 2 — CORE BUILDING BLOCKS

2.1 Script Structure — The Shebang Line

Every shell script must begin with a shebang line. It tells the OS which interpreter to use. Without it, the system may use the wrong shell or fail to execute the script.

```
#!/bin/bash

# This is a comment — the shell ignores any line starting with #
# Goal: Demonstrate basic script structure

echo "Hello from my first script"
```

`#!/bin/bash` → Shebang. The #! tells the OS: use /bin/bash to run this file
`# comment` → Comment — ignored by the shell. Used to document your script
`echo "text"` → Prints text to standard output (the terminal)

⚠ NOTE: Always start every script with #!/bin/bash. This is one of the most fundamental conventions in shell scripting.

2.2 Script Permissions and Execution

A script file must have execute permission before it can be run. chmod adds that permission. Two ways to execute once it is executable.

```
chmod 755 myscript.sh      # Make executable
./myscript.sh              # Execute using current directory path
myscript.sh                # Execute if script is in a PATH directory
```

`chmod 755` → owner=rwx, group=rx, others=rx — standard script permission

`./` → Dot-slash tells the shell: look in the current directory for this file

⚠ NOTE: Without execute permission you get 'Permission denied'. Always chmod 755 your scripts.

2.3 Variables

Variables are named storage locations. Assign a value with = (no spaces). Access the value with $. Wrap in curly braces ${} for clarity — mandatory when combining with other text.

```
MY_VAR='Hello World'       # String assignment
NUMBER=42                  # Number assignment

echo ${MY_VAR}             # Accessing a variable
echo "Value is: ${NUMBER}" # Variable inside double quotes

# Command substitution — store output of a command in a variable
CURRENT_USER=$(id -un)
TODAY=$(date +%F)

# Special built-in shell variables
echo ${UID}                # Current user's numeric ID (read-only)
echo ${HOME}               # Current user's home directory
echo ${RANDOM}             # Random integer 0-32767 (new value each access)
echo ${HOSTNAME}           # Name of this machine
echo $$                    # PID of the current running script
```

`MY_VAR='value'` → Assigns value to MY_VAR. No spaces around = — spaces break the assignment

`${VAR}` → Expands the variable — substitutes its stored value in place

`$(command)` → Command substitution — runs the command and substitutes its output

`$$` → Special variable — PID (Process ID) of the currently running script. Useful for creating unique temp filenames

⚠ NOTE: Never put spaces around = in assignment. MY_VAR = 'value' is WRONG — the shell tries to run a command called MY_VAR. MY_VAR='value' is correct.

2.4 Quoting Rules

Quoting controls whether the shell expands variables and special characters. This is one of the most misunderstood areas of shell scripting and a frequent source of bugs.

Quote Type	Effect	Example	Output
Single quotes ''	Everything literal — NO expansion	'User is $USER'	User is $USER
Double quotes ""	Allows variable and command expansion	"User is $USER"	User is allen
No quotes	Expansion happens AND word splitting occurs on spaces	$VAR with spaces	splits into separate words

$( )	Command substitution — modern syntax, nestable	$(date +%F)	2026-04-02
Backticks ``	Command substitution — legacy syntax, avoid	`date +%F`	2026-04-02

⚠ NOTE: Always use $( ) for command substitution instead of backticks. Backticks are legacy syntax, harder to read, and cannot be nested inside each other.

2.5 Shell Built-ins vs. External Commands

A shell built-in is a command built directly into the bash program. It does not require loading an external file. External commands are programs stored as files on the filesystem under /bin or /usr/bin. The type command tells you which kind a command is. Always prefer built-ins — they are faster and more portable.

```
type echo           # echo is a shell builtin
type ls             # ls is /bin/ls  (external)
type -a echo        # Show ALL instances — builtin AND file

help read           # Get help on a built-in command
help exit
help test

man ls              # Manual page for external commands
```

`type <cmd>` → Tells you: builtin, alias, function, or external program path

`help <builtin>` → Documentation for shell built-ins only — man does not work for built-ins

`man <cmd>` → Manual page for external programs installed on the filesystem

2.6 Standard I/O Streams

Every Linux process has three I/O streams attached by default. Standard input (stdin, fd 0) comes from the keyboard. Standard output (stdout, fd 1) goes to the terminal. Standard error (stderr, fd 2) also goes to the terminal. These can be redirected to files or other processes using redirection operators.

Stream	File Descriptor	Default	Redirect Operator
stdin	0	Keyboard	< (redirect input from file)
stdout	1	Terminal screen	> overwrite, >> append
stderr	2	Terminal screen	2> redirect errors to file

PHASE 3 — SYNTAX AND KEY CONSTRUCTS

3.1 The if Statement

The if statement makes decisions. It evaluates a condition and executes a block of code if true, optionally a different block if false. Conditions are enclosed in double square brackets [[ ]]. The block is always closed with fi (if spelled backwards).

```
#!/bin/bash

# Basic if-then-fi
if [[ ${UID} -eq 0 ]]
then
  echo "You are root."
```

```
fi

# if-then-else-fi
if [[ ${UID} -eq 0 ]]
then
  echo "Running as root."
else
  echo "NOT running as root."
fi

# if-elif-else-fi
if [[ ${UID} -eq 0 ]]
then
  echo "Root."
elif [[ ${UID} -eq 1000 ]]
then
  echo "Normal user UID 1000."
else
  echo "Some other user."
fi
```

`if [[ condition ]]` → Opens the if block. Double brackets [[ ]] is the conditional test syntax in bash
`then` → Required keyword — marks the start of the code to run if condition is true
`else` → Optional — code here runs if the condition is false
`elif [[ condition ]]` → Else-if — tests another condition if all previous conditions were false
`fi` → Closes the if block. Mandatory. Forgetting fi causes a syntax error

Numeric Comparison Operators (for numbers)

Operator	Meaning	Example
-eq	Equal to	[[${UID} -eq 0]]
-ne	Not equal to	[[${UID} -ne 1000]]
-lt	Less than	[[${COUNT} -lt 10]]
-le	Less than or equal	[[${COUNT} -le 10]]
-gt	Greater than	[[${COUNT} -gt 0]]
-ge	Greater than or equal	[[${COUNT} -ge 1]]

String Comparison Operators (for text)

Operator	Meaning	Example
=	Equal (exact match)	[[${NAME} = 'allen']]
!=	Not equal	[[${NAME} != 'root']]
==	Equal with pattern matching on right side	[[${NAME} == all*]]

⚠ NOTE: Use -eq, -ne, -lt, -gt for NUMBERS. Use = and != for STRINGS. Mixing them up gives wrong results without errors in some cases.

File Test Operators

Operator	Meaning
-e file	True if file exists (any type — regular, directory, symlink)
-f file	True if file exists AND is a regular file
-d file	True if file exists AND is a directory
-r file	True if file is readable by the current user
-w file	True if file is writable by the current user
-x file	True if file is executable by the current user
-s file	True if file exists AND has size greater than zero
! -e file	True if file does NOT exist (negate any operator with !)

3.2 Exit Statuses

Every command returns an exit status when it finishes. By convention: 0 = success, non-zero = failure. The special variable $? holds the exit status of the most recently executed command. Your scripts must follow this same convention when they exit.

```
ls /etc
echo "Exit status of ls: ${?}"    # Prints 0 (success)

ls /nonexistent
echo "Exit status of ls: ${?}"    # Prints 2 (failure)

# The exit built-in ends the script with a specific status
exit 0      # Script completed successfully
exit 1      # General failure
exit 2      # Another type of failure (any non-zero works)

# If you omit the exit status, bash uses the status of the last command run
exit        # Exit status = status of whatever ran just before this
```

$? → Special variable holding exit status of the MOST RECENTLY executed command
exit 0 → Exits script signalling success
exit 1 → Exits script signalling failure — any non-zero value means failure

⚠ NOTE: Check $? IMMEDIATELY after the command you care about. Any other command you run first overwrites $? with its own exit status.

3.3 User Input — The read Built-in

The read command pauses the script and waits for the user to type something. It reads one line from standard input and stores it in a named variable. The -p option displays a prompt string. The -s option hides what the user types (for passwords).

```
read -p 'Enter a username: ' USER_NAME

read -s -p 'Enter a password: ' PASSWORD
echo ""    # Move to next line — -s suppresses the newline

read -p 'Enter full name: ' COMMENT

echo "Creating account for: ${USER_NAME}"
```

`read -p 'prompt' VAR` → Displays prompt, waits for input, stores result in VAR

`-s` → Silent — input is not echoed to screen. Always use for passwords

`read VAR1 VAR2` → Splits input on whitespace: first word goes to VAR1, rest to VAR2

3.4 User Account Management Commands

```
# Create a user account
useradd -c '${COMMENT}' -m ${USER_NAME}

# Set password via stdin (pipe the password in)
echo ${PASSWORD} | passwd --stdin ${USER_NAME}

# Force password change on first login
passwd -e ${USER_NAME}

# Delete a user account (leaves home directory)
userdel ${USER_NAME}

# Delete a user AND their home directory
userdel -r ${USER_NAME}
```

`useradd -c 'comment' -m USERNAME` → Creates account. -c sets the full name field. -m creates home directory

`passwd --stdin USERNAME` → Reads the password from stdin — avoids interactive prompt

`passwd -e USERNAME` → Expires the password — forces change on next login

`userdel -r USERNAME` → Deletes user AND removes their home directory. Use with caution

3.5 Generating Random Passwords

```
# Method 1 — $RANDOM (simple, low security)
echo ${RANDOM}

# Method 2 — date + sha256sum (more entropy)
PASSWORD=$(date +%s%N | sha256sum | head -c 48)

# Method 3 — /dev/urandom (most secure, recommended)
PASSWORD=$(cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w 48 | head -n 1)

# Append a special character
SPECIAL=$(echo '!@#$%^&*()-+=' | fold -w1 | shuf | head -c1)
PASSWORD="${PASSWORD}${SPECIAL}"
```

`$RANDOM` → Built-in — returns a new random integer 0-32767 on every access

`date +%s%N` → Epoch seconds + nanoseconds — highly unique each call

`sha256sum` → Cryptographic hash — outputs 64-char hex string

`tr -dc 'a-zA-Z0-9'` → Delete (-d) all chars NOT in the set (-c = complement). Keeps only letters and digits

`fold -w 48` → Wraps output at 48 characters per line

`shuf` → Randomly shuffles lines of input

3.6 For Loops

A for loop iterates over a list and executes a code block for each item. The list can be hardcoded, command output, an array, or the special $@ (all command-line arguments).

```
# Loop over a fixed list
for COLOUR in red green blue
do
  echo "Colour: ${COLOUR}"
done

# Loop over all command-line arguments
```

```
for USER in "$@"
do
  echo "Processing: ${USER}"
done

# Loop over files matching a pattern
for FILE in /etc/*.conf
do
  echo "Config: ${FILE}"
done

# Loop over array elements
SERVERS=("192.168.1.1" "192.168.1.2" "192.168.1.3")
for SERVER in "${SERVERS[@]}"
do
  echo "Connecting to: ${SERVER}"
done
```

`for VAR in list` → Assigns each item in list to VAR one at a time

`do` → Marks the beginning of the code block that runs each iteration

`done` → Closes the for loop — mandatory

`"$@"` → All positional parameters as separate quoted words — safest way to loop over arguments

`"${SERVERS[@]}"` → All elements of the SERVERS array as separate quoted words

3.7 While Loops

A while loop continues executing as long as its condition evaluates to true (exit status 0). Unlike a for loop, a while loop does not iterate over a pre-defined list. It keeps going until the condition becomes false. Essential for processing an unknown number of inputs or running indefinitely.

```
# Basic while loop — count down
X=3
while [[ ${X} -gt 0 ]]
do
  echo "X is: ${X}"
  X=$((X - 1))
done

# Infinite loop — use Ctrl+C to break out
while true
do
  echo "Running... press Ctrl+C to stop"
  sleep 1
done

# Loop through positional parameters using shift
while [[ ${#} -gt 0 ]]
do
  echo "Processing argument: ${1}"
  shift
done

# Read piped input line by line (used in log parsing scripts)
cat /etc/passwd | while read LINE
do
  echo "Line: ${LINE}"
done

# Read pipe with multiple variables (splits on whitespace)
sort /etc/passwd | uniq -c | while read COUNT VALUE
do
  echo "Count: ${COUNT}  Value: ${VALUE}"
```

```
done
```

`while [[ condition ]]` → Checks condition before every iteration. If false from the start, the body never runs

`true` → Built-in that always returns exit status 0 — used to create infinite loops

`sleep 1` → Pauses execution for 1 second. Suffixes: s=seconds, m=minutes, h=hours, d=days

`while read LINE` → Reads one line at a time from stdin (pipe output into it)

`while read COUNT VALUE` → Splits each line on whitespace: first word to COUNT, rest to VALUE

⚠ NOTE: If the while condition never becomes false, you have an infinite loop. Break out with Ctrl+C. When using while with $#, always shift inside the loop — otherwise $# never decreases and the loop runs forever.

3.8 Positional Parameters and the shift Command

When you run a script, arguments supplied on the command line are stored in positional parameters $1, $2, $3 etc. The script name itself is in $0.

```
# Run as: ./demo.sh apple bread candy

echo "Script name:    ${0}"    # demo.sh
echo "First arg:      ${1}"    # apple
echo "Second arg:     ${2}"    # bread
echo "All arguments:  ${@}"    # apple bread candy (separate words)
echo "All as string:  ${*}"    # apple bread candy (single string)
echo "Argument count: ${#}"    # 3
```

Parameter	Meaning
${0}	Name of the script itself
${1}, ${2}, ...	First, second positional argument
${@}	All arguments as separate quoted words — preferred in loops
${*}	All arguments as a single string when quoted
${#}	Total number of arguments passed to the script

The shift Command

shift removes $1 from the parameter list, shifts all remaining parameters down by one, and decreases $# by one. $2 becomes $1, $3 becomes $2, and so on. Used in while loops to process arguments one at a time.

```
# Before shift:  $1=apple  $2=bread  $3=candy  $#=3
shift
# After shift:   $1=bread  $2=candy  $#=2

# Practical pattern — grab username, treat rest as comment
USER_NAME=${1}
shift
COMMENT=${@}    # Everything left after shift = the comment
```

3.9 The case Statement

The case statement is a multi-way branch. It compares a value against multiple patterns and runs the first matching block. It is cleaner than a long if-elif-elif chain when handling many specific values. The structure closes with esac (case spelled backwards).

```
#!/bin/bash
```

```
case ${1} in
  start)
    echo "Starting the service..."
    ;;
  stop)
    echo "Stopping the service..."
    ;;
  restart|reload)
    echo "Restarting or reloading..."
    ;;
  *)
    echo "Usage: ${0} start|stop|restart|reload"
    exit 1
    ;;
esac
```

`case ${VAR} in` → Opens the case block — compares value of VAR against each pattern

`pattern)` → Each case ends with a closing parenthesis

`pattern1|pattern2)` → Pipe separates multiple patterns that trigger the same block

`;;` → Ends the case block. Required after each case — like a break

`*)` → Wildcard catch-all — matches anything not matched above

`esac` → Closes the case statement. Mandatory

PHASE 4 — I/O REDIRECTION, PIPES, AND DATA PROCESSING

4.1 I/O Redirection

```
echo 'Hello' > output.txt          # Redirect stdout — OVERWRITE
echo 'World' >> output.txt         # Redirect stdout — APPEND
sort < names.txt                   # Feed file as stdin to command
ls /badpath 2> errors.txt          # Redirect stderr to file
ls /badpath > all.txt 2>&1         # Redirect both stdout AND stderr to same file
ls /badpath &> all.txt             # Shorthand for same thing
ls /badpath > /dev/null 2>&1       # Discard ALL output
echo "Error occurred" >&2          # Send stdout TO stderr (for error messages)
```

`>` → Redirect stdout to file. Creates file if missing, OVERWRITES if it exists

`>>` → Redirect stdout to file. Creates if missing, APPENDS if it exists

`<` → Feed a file as stdin to a command

`2>` → Redirect stderr (file descriptor 2) to a file

`2>&1` → Redirect stderr to the same place as stdout. The & signals file descriptor

`>&2` → Redirect stdout to stderr — use this for all error messages in scripts

`/dev/null` → The null device — discards everything sent to it. Like a black hole

⚠ NOTE: Error messages belong on stderr, NOT stdout. Always use echo 'error' >&2 for error output in your scripts. This allows callers to separate normal output from errors.

4.2 Pipes

A pipe connects stdout of one command to stdin of the next. It chains commands together, feeding output of one as input to the next. This is the foundation of Unix philosophy: small tools combined to do complex things.

```
cat /etc/passwd | grep bash                # Basic pipe
cat /etc/passwd | grep bash | cut -d: -f1 | sort   # Chained pipes
command 2>&1 | grep error                  # Pipe both stdout and stderr
command |& grep error                      # Shorthand for above
```

`|` → Pipe — stdout of left command becomes stdin of right command

`|&` → Pipe both stdout AND stderr as stdin to the next command

4.3 Key Text Processing Tools

cut — Extract Columns

```
cut -d: -f1 /etc/passwd      # Field 1, colon-delimited
cut -d: -f7 /etc/passwd      # Field 7 (the shell field)
cut -d' ' -f2 file.txt       # Field 2, space-delimited
cut -c1-5 file.txt           # Characters 1 through 5 of each line
```

`-d:` → Delimiter — use colon as field separator

`-f1` → Field number to extract

sort — Sort Lines

```
sort names.txt               # Alphabetical sort (default)
sort -n numbers.txt          # Numeric sort (2 before 10)
sort -r names.txt            # Reverse sort
sort -rn numbers.txt         # Numeric reverse sort
sort -u names.txt            # Sort AND remove duplicates
sort -t: -k3 -n /etc/passwd  # Sort on field 3, colon-delimited, numerically
```

`-n` → Numeric sort — treats values as numbers not strings

`-r` → Reverse the sort order

`-t:` → Field delimiter for sorting (colon here)

`-k3` → Sort key — sort on field number 3

`-u` → Unique — removes duplicate lines while sorting

uniq — Remove or Count Duplicate Lines

```
sort names.txt | uniq        # Remove consecutive duplicates (sort first!)
sort names.txt | uniq -c    # Count occurrences of each unique line
sort names.txt | uniq -d    # Show only lines that ARE duplicated
```

⚠ NOTE: uniq only removes ADJACENT duplicate lines. Always sort before piping to uniq or duplicates that are not next to each other will not be removed.

grep — Search for Patterns

```
grep bash /etc/passwd        # Lines containing 'bash'
grep -i 'error' logfile.txt  # Case-insensitive match
grep -c 'bash' /etc/passwd   # Count matching lines
grep -v 'root' /etc/passwd   # Invert — lines that do NOT match
grep Failed syslog.txt       # Literal word search
```

awk — Column-Based Processing

```
awk '{print $1}' file.txt         # Print first column (space-delimited)
awk '{print $1, $3}' file.txt    # Print columns 1 and 3
awk -F: '{print $1}' /etc/passwd  # Colon as field separator
awk '{print $NF}' file.txt        # Print LAST column (NF = number of fields)
awk '{print $(NF-3)}' file.txt    # 4th column from the end
awk -F', ' '{print $2}' file.txt  # Comma-space as separator

# BEGIN block — runs before processing any lines
awk 'BEGIN{print "Count,IP,Location"} {print $0}' file.txt
```

`$1, $2 ...` → Field variables — first, second field on the current line

`NF` → Built-in variable — total Number of Fields on the current line

`BEGIN{ }` → Code in BEGIN block runs once before processing any lines — used for headers

wc, head, tail

```
wc -l /etc/passwd      # Count lines
wc -w file.txt         # Count words
wc -c file.txt         # Count bytes
```

```
head -5 file.txt       # First 5 lines
tail -5 file.txt       # Last 5 lines
tail -f /var/log/syslog  # Follow — watch file grow in real-time
```

4.4 sed — Stream Editor

sed stands for Stream Editor. It processes text line by line and performs transformations without opening an interactive editor. The most common use is find-and-replace. By default sed does NOT modify the original file — it sends transformed output to stdout.

```
sed 's/old/new/' file.txt        # Replace FIRST occurrence per line
sed 's/old/new/g' file.txt       # Replace ALL occurrences (global)
sed 's/old/new/i' file.txt       # Case-insensitive replacement
sed 's/old/new/gi' file.txt      # Global + case-insensitive
sed -i 's/old/new/g' file.txt    # In-place edit — modifies the actual file
sed '/pattern/d' file.txt        # Delete lines matching pattern
sed '/^\$/d' file.txt           # Delete blank lines
sed 's|/old/path|/new/path|g' f  # Use | as delimiter (useful when text has /)
```

`s/search/replace/` → Substitution — find search, replace with replacement, forward slashes are delimiters

`/g` → Global flag — replace ALL occurrences on each line, not just the first

`/i` → Case-insensitive flag

`-i` → In-place flag — modifies the file directly

`/pattern/d` → Delete all lines matching pattern

⚠ NOTE: Without /g, sed only replaces the FIRST occurrence per line. Always add /g unless you specifically want just the first match.

PHASE 5 — FUNCTIONS, LOGGING, AND ADVANCED FEATURES

5.1 Functions — Full Coverage

A function is a named block of code you define once and call many times. Functions implement the DRY principle (Don't Repeat Yourself). Functions must be defined BEFORE they are called — always place them at the top of the script. There are two valid syntaxes.

```
#!/bin/bash

# —— Syntax 1: name() { } ——
greet() {
  echo "Hello from greet function"
}

# —— Syntax 2: function name { } (no parentheses) ——
function greet {
  echo "Hello from greet function"
}

# Both syntaxes are identical in behaviour.
# Syntax 1 (with parentheses) is more widely used.

# Call a function — just use its name (NO parentheses when calling)
greet
```

`function_name() { }` → Function definition. Curly braces contain the function body

`function function_name { }` → Alternative syntax — keyword 'function' replaces the parentheses

`greet` → Calling a function — just write its name as a command. Never use () when calling

⚠ NOTE: Define ALL functions at the TOP of the script before any code that calls them. Bash reads top-to-bottom — calling a function before it is defined causes 'command not found'.

Passing Arguments to Functions

Functions accept arguments just like scripts do. Inside a function, $1 is the first argument passed TO the function, $2 is the second, and $@ is all of them. Note: $0 is still the script name even inside a function.

```
log() {
  local MESSAGE=${@}      # ${@} = all arguments passed to this function
  echo "${MESSAGE}"
}

log 'Starting the script'
log 'Creating user account'

# Function with a specific first argument
backup_file() {
  local ORIGINAL=${1}     # First argument = path of file to back up
  local BACKUP_DIR="/var/tmp"
  local BACKUP_NAME
  BACKUP_NAME="$(basename ${ORIGINAL}).$(date +%F.%N)"
  cp -p ${ORIGINAL} ${BACKUP_DIR}/${BACKUP_NAME}
}

backup_file /etc/passwd
```

`local MESSAGE=${@}` → local declares a variable scoped to this function only. ${@} = all function arguments

`cp -p` → Preserve — copies the file AND preserves its original timestamps and permissions

`date +%F` → Outputs date in YYYY-MM-DD format — great for sortable filenames

`date +%F.%N` → Date + nanoseconds — ensures unique backup filename even for same-day backups

local — Variable Scope

By default ALL variables in bash are global — visible everywhere in the script including inside functions. Using the local keyword restricts a variable's scope to the function where it is declared. This prevents accidental overwriting of same-named variables elsewhere in the script.

```
MY_VAR='global'

my_function() {
  local MY_VAR='local'    # Only visible inside this function
  echo "Inside function: ${MY_VAR}"   # Prints: local
}

my_function
echo "Outside function: ${MY_VAR}"    # Prints: global  (unchanged)
```

⚠ NOTE: Best practice: always declare variables inside functions with 'local'. Without it, the function can accidentally change a global variable and create a very hard-to-find bug.

return vs. exit Inside Functions

This is a critical distinction. exit always exits the ENTIRE script regardless of whether it is called inside a function or not. return exits only the function and passes control back to the caller. You can supply a return status just like an exit status — 0 for success, non-zero for failure.

Command	Where it stops	Use case
exit 0 / exit 1	Entire script terminates	Use at the top level to end the script

return 0 / return 1	Only the function returns	Use inside functions to signal success/failure
return (no value)	Function returns with exit status of last command	Let the last command's status bubble up

```
check_file() {
  local FILE=${1}
  if [[ ! -e ${FILE} ]]
  then
    echo "File not found: ${FILE}" >&2
    return 1    # Return FROM the function with failure status
  fi
  return 0      # Return FROM the function with success status
}

check_file /etc/passwd
if [[ ${?} -eq 0 ]]
then
  echo "File check passed"
fi
```

⚠ NOTE: Inside a function use return, not exit. Using exit inside a function will terminate your ENTIRE script, not just the function — this is a very common bug.

readonly — Constant Variables

The readonly built-in makes a variable unchangeable for the duration of the script. Once set as readonly, no code — inside or outside functions — can modify it. This is bash's equivalent of a constant in other programming languages.

```
# Method 1 — declare and mark readonly at the same time
readonly VERBOSE=true

# Method 2 — assign first, then lock it
LOG_FILE='/var/log/myscript.log'
readonly LOG_FILE

# Trying to change a readonly variable causes an error
VERBOSE=false   # ERROR: bash: VERBOSE: readonly variable
```

`readonly VARNAME=value` → Declares variable AND marks it immutable in one line

`readonly VARNAME` → Marks an already-assigned variable as immutable going forward

5.2 Logging with logger and the log() Pattern

The logger command is an external utility (not a built-in) that writes messages to the Linux system log — /var/log/messages on RHEL/CentOS systems. This is part of the syslog system. Using logger in production scripts creates a permanent audit trail that survives even if the script's output is discarded.

```
# Basic logger usage
logger 'Hello from my script'

# Add a tag so log entries are identified by script name
logger -t my-script "Backup started"

# View the syslog to confirm
sudo tail /var/log/messages

# Full log() function — writes to both syslog AND stdout (if verbose)
readonly VERBOSE=true
```

```
log() {
  local MESSAGE=${@}
  if [[ ${VERBOSE} = 'true' ]]
  then
    echo ${MESSAGE}
  fi
  logger -t ${0} ${MESSAGE}   # Always write to syslog regardless of verbose
}

log 'Script started'
log 'Processing user accounts'
```

`logger 'message'` → Writes message to system log (/var/log/messages on RHEL, /var/log/syslog on Ubuntu)

`logger -t my-script` → -t adds a tag — identifies which script or program wrote the log entry

`${VERBOSE} = 'true'` → Check a global flag before deciding whether to print to screen

⚠ NOTE: Use logger in any production script. If your script runs as a cron job with output discarded, logger is the only way to know what happened. The syslog timestamp format is: date hostname tag: message.

5.3 getopts — Parsing Command-Line Options

getopts is a shell built-in that processes command-line options in the standard Unix style (-v, -l, -s). It is always used inside a while loop combined with a case statement. getopts processes one option at a time, sets a variable to the current option letter, and sets OPTARG to the option's value if one is required. If an option requires an argument, follow that letter with a colon in the optstring.

```
#!/bin/bash

LENGTH=48
VERBOSE=false
USE_SPECIAL=false

usage() {
  echo "Usage: ${0} [-v] [-l LENGTH] [-s]" >&2
  echo "  -v  Verbose mode" >&2
  echo "  -l  Password length (default: 48)" >&2
  echo "  -s  Append a special character" >&2
  exit 1
}

log() {
  local MSG=${@}
  [[ ${VERBOSE} = 'true' ]] && echo ${MSG}
}

# getopts loop
while getopts vl:s OPTION
do
  case ${OPTION} in
    v) VERBOSE=true ;;
    l) LENGTH=${OPTARG} ;;
    s) USE_SPECIAL=true ;;
    ?) usage ;;
  esac
done

# Remove processed options from positional parameters
shift $((OPTIND - 1))
```

```
# If any extra arguments remain — show usage
[[ ${#} -gt 0 ]] && usage

log 'Generating password...'
PASSWORD=$(cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w ${LENGTH} | head -n 1)

if [[ ${USE_SPECIAL} = 'true' ]]
then
  SPECIAL=$(echo '!@#$%^&*()-+=' | fold -w1 | shuf | head -c1)
  PASSWORD="${PASSWORD}${SPECIAL}"
fi

log 'Done.'
echo ${PASSWORD}
exit 0
```

`while getopts vl:s OPTION` → optstring = 'vl:s'. v and s take no argument. l: means -l REQUIRES a value after it

`${OPTARG}` → Set by getopts — holds the argument value for options that require one (e.g. -l 48)

`${OPTIND}` → Set by getopts — index of the next argument AFTER all options have been processed

`shift $((OPTIND - 1))` → Strips all processed options from $@ leaving only non-option arguments

`?)` → Catches any unrecognised option that getopts rejects

5.4 Arithmetic Expansion and Evaluation

Bash provides built-in integer arithmetic using the $(( )) syntax. Inside double parentheses you write math expressions. The shell evaluates the expression and substitutes the result. Bash only supports integers — decimals are truncated, not rounded.

```
$((1 + 2))       # Addition       → 3
$((10 - 3))      # Subtraction    → 7
$((4 * 5))       # Multiplication → 20
$((10 / 3))      # Integer division → 3  (NOT 3.33, decimal truncated)
$((10 % 3))      # Modulo (remainder) → 1

COUNT=0
((COUNT++))       # Increment by 1
((COUNT+=5))      # Add 5 to COUNT
((COUNT-=2))      # Subtract 2 from COUNT

# Variables inside (( )) do NOT need $ prefix
DICEA=3  DICEB=6
TOTAL=$((DICEA + DICEB))    # TOTAL = 9

# Floating point — use bc (external calculator)
echo '6/4' | bc -l          # Returns 1.5

# let built-in (alternative syntax)
let RESULT=2+3              # RESULT = 5
let RESULT++                # RESULT = 6
```

`$(( ))` → Arithmetic expansion — evaluates expression and substitutes the result as a value

`(( ))` → Arithmetic evaluation — executes arithmetic but does NOT produce output. Used for increment/change

`%` → Modulo — returns the remainder of integer division. 10%3=1

`bc -l` → External basic calculator with math library. Handles floating point

⚠ NOTE: Bash does NOT support floating point. 6/4 in Bash returns 1, not 1.5. Use bc or awk when you need decimal precision.

5.5 Finding Commands Not in PATH — locate, find, which

Sometimes a command exists on the system but is not in your current PATH, so type and which cannot find it. This is common for system administration commands in /usr/sbin which are only in root's PATH. Three tools help you locate files on the filesystem.

```
# which — searches directories in your current PATH
which userdel     # May return 'not found' for sbin commands

# locate — fast search using a pre-built database index
locate userdel
locate userdel | grep bin   # Narrow to binary directories

# Update the locate database (needs root, run if locate misses new files)
sudo updatedb

# find — real-time recursive search (slower but always current)
find /usr/sbin -name userdel          # Search specific directory
find / -name userdel 2>/dev/null      # Search whole filesystem, discard errors
sudo find / -name userdel             # With root — finds files in protected dirs

# sudo !! — re-run the previous command with sudo
sudo !!
```

`which` → Searches your current PATH and shows the full path to the command

`locate` → Searches a pre-built database index. Very fast but not real-time (updated daily by updatedb)

`sudo updatedb` → Forces an immediate rebuild of the locate index. Needs root

`find / -name pattern` → Real-time recursive search. Slower than locate but always current

`2>/dev/null` → Discards all stderr output — hides 'Permission denied' messages from find

`sudo !!` → Re-runs the previous command prefixed with sudo

⚠ NOTE: System admin commands (userdel, useradd, etc.) are in /usr/sbin which is only in root's PATH. As a regular user, use the full path: /usr/sbin/userdel, or switch to root first.

PHASE 6 — INTERVIEW FAVOURITES AND REAL-WORLD PATTERNS

6.1 Shell Script Best Practices — The Conventions

```
Convention 1   Always start with #!/bin/bash
Convention 2   Variable names in UPPERCASE to distinguish from commands
Convention 3   Declare local variables inside functions with the 'local'
keyword
Convention 4   Exit 0 at end of successful script. Exit non-zero on
failure
Convention 5   Send error messages to stderr:  echo 'error' >&2
Convention 6   Define all functions at the TOP before any calling code
Convention 7   Provide a usage() function for scripts that accept options
Convention 8   Name scripts with .sh extension for clarity
Convention 9   Add a comment header describing purpose and usage
Convention 10  Use chmod 755 to make scripts executable
Convention 11  Use 'local' for all function variables — avoid global side
effects
Convention 12  Use readonly for constants that should never change
Convention 13  Use logger in production scripts for permanent audit trails
Convention 14  Use return inside functions, exit at the script top level
only
```

6.2 Debugging Shell Scripts

```
# Method 1 — Run with -x flag (xtrace mode)
bash -x myscript.sh

# Method 2 — Enable xtrace inside the script
set -x       # Turn ON xtrace from this point
# ... commands to trace ...
set +x       # Turn OFF xtrace  (+ disables the option)

# Method 3 — Exit immediately on any error
#!/bin/bash
set -e       # Exit script the moment any command returns non-zero

# Method 4 — Combine both for maximum safety
set -xe
```

`bash -x script` → Runs the script in xtrace — prints each command before executing it (shows variable values)

`set -x / set +x` → Toggle xtrace on (+x enables something despite the confusing name, +x disables)

`set -e` → Exit immediately if any command fails — prevents scripts running past errors

6.3 Complete Script — User Account Creator (Interactive)

```
#!/bin/bash
# Purpose: Create a user account interactively.
# Must be run as root.

if [[ ${UID} -ne 0 ]]
then
  echo 'Please run with sudo or as root.' >&2
  exit 1
fi

read -p 'Enter username: ' USER_NAME
read -p 'Enter real name: ' COMMENT
read -p 'Enter password: ' PASSWORD

useradd -c "${COMMENT}" -m ${USER_NAME}

if [[ ${?} -ne 0 ]]
then
  echo 'Account creation failed.' >&2
  exit 1
fi

echo ${PASSWORD} | passwd --stdin ${USER_NAME}
passwd -e ${USER_NAME}

echo "Username: ${USER_NAME}"
echo "Password: ${PASSWORD}"
echo "Host:     $(hostname)"
exit 0
```

6.4 Complete Script — User Account Creator (Args + Auto Password)

```
#!/bin/bash
# Purpose: Create a user from command-line arguments with auto-generated password.
# Usage: sudo ./add-new-local-user.sh USERNAME [COMMENT...]

if [[ ${UID} -ne 0 ]]
then
  echo 'Run as root.' >&2
```

```
  exit 1
fi

if [[ ${#} -lt 1 ]]
then
  echo "Usage: ${0} USER_NAME [COMMENT]..." >&2
  echo 'Create a new local user account.' >&2
  exit 1
fi

USER_NAME=${1}
shift
COMMENT=${@}

PASSWORD=$(date +%s%N${RANDOM}${RANDOM} | sha256sum | head -c 48)

useradd -c "${COMMENT}" -m ${USER_NAME} &>/dev/null

if [[ ${?} -ne 0 ]]
then
  echo "Failed to create ${USER_NAME}." >&2
  exit 1
fi

echo ${PASSWORD} | passwd --stdin ${USER_NAME} &>/dev/null
passwd -e ${USER_NAME} &>/dev/null

echo "Username: ${USER_NAME}"
echo "Password: ${PASSWORD}"
echo "Host:     $(hostname)"
exit 0
```

`&>/dev/null` → Redirect both stdout AND stderr to /dev/null — suppresses all output from that command

6.5 Complete Script — Password Generator with getopts + Functions

```
#!/bin/bash
# Usage: ./mkpasswd.sh [-v] [-l LENGTH] [-s]

readonly LENGTH_DEFAULT=48
LENGTH=${LENGTH_DEFAULT}
VERBOSE=false
USE_SPECIAL=false

usage() {
  echo "Usage: ${0} [-v] [-l LENGTH] [-s]" >&2
  exit 1
}

log() {
  local MSG=${@}
  [[ ${VERBOSE} = 'true' ]] && echo ${MSG}
}

while getopts vl:s OPT
do
  case ${OPT} in
    v) VERBOSE=true ;;
    l) LENGTH=${OPTARG} ;;
    s) USE_SPECIAL=true ;;
    ?) usage ;;
  esac
done
```

```
shift $((OPTIND - 1))
[[ ${#} -gt 0 ]] && usage

log "Generating ${LENGTH}-char password..."
PASSWORD=$(cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w ${LENGTH} | head -n 1)

if [[ ${USE_SPECIAL} = 'true' ]]
then
  log 'Appending special character...'
  SPECIAL=$(echo '!@#$%^&*()-+=' | fold -w1 | shuf | head -c1)
  PASSWORD="${PASSWORD}${SPECIAL}"
fi

log 'Done.'
echo ${PASSWORD}
exit 0
```

6.6 Complete Script — Log File Analyser (Pipes + awk + geoiplookup)

```
#!/bin/bash
# Purpose: Find IP addresses with > LIMIT failed SSH login attempts.
# Usage: ./show-attackers.sh <logfile>

LIMIT=10

if [[ ! -e ${1} ]]
then
  echo "Cannot open ${1}." >&2
  exit 1
fi

echo 'Count,IP,Location'

grep Failed ${1} | awk -F 'from ' '{print $2}' | awk '{print $1}' | \
  sort | uniq -c | sort -rn | \
  while read COUNT IP
  do
    if [[ ${COUNT} -gt ${LIMIT} ]]
    then
      LOCATION=$(geoiplookup ${IP} | awk -F ', ' '{print $2}')
      echo "${COUNT},${IP},${LOCATION}"
    fi
  done

exit 0
```

`-e ${1}` → File test — true if the path exists

`! -e` → Negate — true if path does NOT exist

`while read COUNT IP` → Read each piped line, split into two variables on whitespace

`\` → Line continuation — the command continues on the next line

`geoiplookup ${IP}` → External command — returns geographic location of an IP address

6.7 Key Interview Q&A

Q: What is the difference between $@ and $*?

When UNQUOTED both expand the same. The difference only appears when QUOTED. "$@" expands each parameter as a separate quoted word — spaces within arguments are preserved. "$*" treats all parameters as a single string. Always use "$@" in loops when arguments may contain spaces.

Q: What is the difference between > and >>?

> creates a new file or OVERWRITES an existing file completely. >> creates a new file or APPENDS to an existing file. Use >> for log files. Use > when you want a fresh output file on each run.

Q: What does 2>&1 mean?

It redirects file descriptor 2 (stderr) to the same destination as file descriptor 1 (stdout). The & signals that the following number is a file descriptor, not a filename. This combines both streams into one.

Q: What is the difference between return and exit?

exit terminates the ENTIRE script regardless of where it is called — even inside a function. return exits only the current FUNCTION and passes control back to the caller. Always use return inside functions and exit at the script's top level.

Q: Why use local inside a function?

Without local, all bash variables are global by default. A function without local variables can accidentally overwrite a same-named variable defined elsewhere in the script, creating very hard-to-debug side effects. Using local limits the variable to the function's scope.

Q: What does readonly do?

readonly makes a variable immutable — it cannot be changed anywhere in the script after being set. Attempting to change it causes an error. It is bash's version of a constant. Use it for configuration values that should never change during a script's run.

Q: What is the logger command?

logger is an external command that writes messages to the Linux syslog system, which persists to /var/log/messages (RHEL) or /var/log/syslog (Ubuntu). Use it in production scripts so there is always a permanent audit trail even when a script's terminal output is discarded or the script runs as a cron job.

BONUS — ANSIBLE-LIKE SSH DEPLOYMENT SCRIPTS

B.1 Concept — What This Script Does

This is a real-world DevOps script that demonstrates how automation tools like Ansible work internally. The script takes a list of remote machine IP addresses, connects to each one via SSH using key-based authentication, and installs nginx on every machine. No passwords are typed — SSH keys handle authentication automatically.

Pre-requisite — SSH Key Distribution (done once manually)

Before running either deployment script, you must copy your SSH public key to each target machine. This is the only manual step. After this, everything is automated.

```
# Generate an SSH key pair if you don't have one
ssh-keygen -t rsa -b 4096

# Copy your public key to each target machine (enter password once per machine)
ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@192.168.1.101
ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@192.168.1.102
ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@192.168.1.103
```

```
ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@192.168.1.104
ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@192.168.1.105

# After this, SSH connects without a password prompt
ssh ubuntu@192.168.1.101   # Logs in directly — no password needed
```

`ssh-keygen -t rsa -b 4096` → Generates a 4096-bit RSA key pair: private key (~/.ssh/id_rsa) and public key (~/.ssh/id_rsa.pub)

`ssh-copy-id` → Appends your public key to the remote machine's ~/.ssh/authorized_keys file

⚠ NOTE: ssh-copy-id is the ONE manual step. Ansible faces the same requirement — you must distribute SSH keys before any automation can happen. In Azure, this step is eliminated by providing the public key at VM creation time in the portal or via Terraform.

B.2 New Concepts Introduced in the Scripts

Concept	Syntax	Meaning
Bash array	HOSTS=("ip1" "ip2")	Stores multiple values. Access with ${HOSTS[@]}
Array length	${#HOSTS[@]}	Number of elements in the HOSTS array
Array loop	for H in "${HOSTS[@]}"	Iterates over every element in the array
Here Document	<< 'MARKER' ... MARKER	Sends a multi-line block as stdin. Single-quoted marker = no local expansion
Background job	function_name &	Runs the function/command in background — shell does NOT wait
Last bg PID	$!	PID of the most recently launched background job
PID array	PIDS+=("${!}")	Appends the background job's PID to the PIDS array
Wait for jobs	wait "${PIDS[@]}"	Pauses main script until all listed PIDs finish
ANSI colours	\033[0;32m	Terminal escape codes for green/red/yellow output

The Here Document — Key Concept

A here document sends a block of text as stdin to a command. In the deployment scripts, the entire set of remote commands is written as a here document and fed to bash running on the remote machine over SSH. The single quotes around the marker name prevent local variable expansion — all dollar signs are evaluated on the REMOTE machine, not on the control machine.

```
ssh user@remotehost 'bash -s' << 'REMOTE_SCRIPT'
  # Everything here runs ON THE REMOTE MACHINE
  echo "I am running on: $(hostname)"
  sudo apt-get install -y nginx
REMOTE_SCRIPT

# Compare: without single quotes, $(hostname) would run on LOCAL machine
ssh user@remotehost 'bash -s' << REMOTE_SCRIPT
  echo "I am running on: $(hostname)"   # hostname of LOCAL machine inserted here
REMOTE_SCRIPT
```

`'bash -s'` → Tells the remote machine's bash: read commands from stdin (the here document)

`<< 'MARKER'` → Here document. Single-quoted MARKER = all $ are evaluated on REMOTE. No quotes = local expansion

Sequential vs. Parallel — The Core Difference

Aspect	Sequential	Parallel
How jobs run	One at a time — next starts when previous finishes	All 5 launch simultaneously with &
Total time	Sum of all individual times	Roughly equal to the slowest single machine
Output	Printed to terminal immediately in order	Written to per-host log files (avoids mixed output)
$! variable	Not needed	Captures PID of each background job
wait command	Not needed	Essential — main script waits for all PIDs to finish
Debugging	Easier — one machine at a time	Harder — check per-host log files
Production use	Suitable for small sets (<5 hosts)	Preferred for large sets (10+ hosts)

The & and wait Pattern (Parallel Core Logic)

```
PIDS=()   # Empty array to collect background job PIDs

for HOST in "${HOSTS[@]}"
do
  deploy_on_host "${HOST}" &   # & = launch in background, do NOT wait
  PIDS+=("${!}")               # ${!} = PID of the job just launched
done

# Main script pauses here until ALL background jobs complete
wait "${PIDS[@]}"

# Only reaches here after all 5 machines are done
echo 'All deployments finished.'
```

`command &` → The & launches the command as a background process and immediately returns control to the shell

`$!` → Holds the PID of the most recently backgrounded job. Captured right after & to remember each job's ID

`PIDS+=(value)` → Append to array syntax. Each background job's PID is added to the PIDS array

`wait "${PIDS[@]}"` → Blocks here until all listed PIDs finish. Without this the script prints the summary before jobs complete

END OF NOTES — ALL PHASES COMPLETE

These notes cover the complete shell scripting course across all six phases plus the bonus deployment script section. Every concept from the Udemy transcript is included: from the shebang line and basic variables, through loops, functions, I/O redirection, text processing, getopts, and logging, all the way to real-world deployment automation. The study approach for interviews: understand exit statuses deeply, understand I/O streams deeply, and be able to explain any line of code shown to you by walking through what each keyword and operator does.